VU AI HUB · LECTURE NOTES

Applied Programming for AI

Year 1, June 2026

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/3729801a9309804db093fb91d1fb29e0

Contents

1. 1st of June (HC)
2. 5th of June (WG)
3. 8th of June (HC)
4. 15th of June (HC)

1st of June (HC)

GitHub

Repositories

- A project folder managed by Git
- Tracks all changes to files over time
- Contains the entire history of the project
- Can be local (on your machine) or remote (on platforms like GitHub)
  - So Git =/= GitHub
  - Every team member will have a local repository of their own and one shared remote repository

Commits

- A snapshot of your files at a specific point in time
- A way to save your progress with a message describing the change
- Recorded history of what changed, who changed it, and when
  - ex. like a save point in a game, you can always go back to that saved point

Branches

- A parallel version of your codebase
- Lets you work on features, fixes, or experiments without changing the main code
- Can be merged back into the main line when ready
- You will do most of your work in personal branches
  - Each repository can have its own branches

Merging

- The process of combining two branches together
  - While collaborating this is often your local version of a branch with the remote version
- Typically done when a feature is ready to be added back to the main branch
- Git tries to auto-resolve differences, but sometimes you'll need to help
  - Sometimes changes cause conflicts

Basic Commands Git

- git clone [url] - copy a remote repository
- git add [files] - stages new files or modified files
- git commit - creates a new commit
  - commits always need a message (git commit -m “some message”)

Pushing and pulling

- git push [remote][branch] - update the remote branch with your local commits
- git fetch [remote] - Download the state of the remote
  - this does not make any changes to your files
- git pull [remote][branch] - merge the remote branch into your current branch
  - this does the fetch first and then the pull

*If you cloned your repository the remote repository is called origin

Branches

- git branch[name] - Create a branch with a certain name
- git switch[name] - Switch to work in a certain branch
  - git switch -c [name] creates and switched to the branch
- git merge [branch] - Merge the target branch into the current
  - this is also subject to merge conflicts

*Remote branches and your local branches are not the same

*You merge into the one you want to keep (not the other way around)

Your Project

- Remote repository hosted on GitHub
- main branch that will always contain a working version of the project
- dev branch for the team to collaborate on releases to main
- Personal remote branches for each team member (remote)
  - push your work daily to your personal branch
  - you can have multiple personal branches for each feature
- Every project needs a README.md
  - besides an introduction to the project, it should also contain installation instructions
  - also a requirements.txt

5th of June (WG)

Version Control System (Git)

The Git Basics

- `git status` — see what's changed
- `git add .` — stage all changes
- `git commit -m "message"` — save a snapshot
- `git push` — send to GitHub
- `git pull` — get latest from GitHub

Branches

- `git checkout -b branch-name` — create and switch to a new branch
- `git checkout main` — switch back to main
- `git merge branch-name` — merge a branch into your current one
- `git branch -a` — see all branches (local + remote)

Always `git pull` before you `git push` — otherwise you'll get the rejection error you saw in Task 3.

Merge Conflicts

They happen when two people edit the same line. Git marks them like this:

<<<<<<< HEAD
your change
=======
their change
>>>>>>> branch-name

You manually pick what to keep, delete the markers, then `git add` and `git commit`.

.gitignore

Never commit secrets (API keys, passwords). Always add them to `.gitignore` so Git ignores them completely.

RepoA vs RepoB lesson

This simulated working in a team — you and a teammate both editing the same code. In real life, each teammate has their own clone, which is why pulling before pushing matters so much.

8th of June (HC)

REST API

Representational State Transfer

- The communication is stateless, every message contains all the necessary information

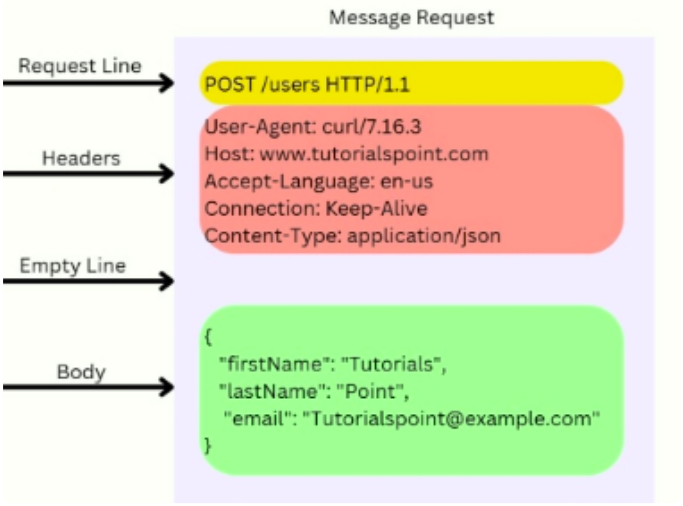
HTTP requests match up with CRUD operations:

GET → Retrieve data

POST → Create new data

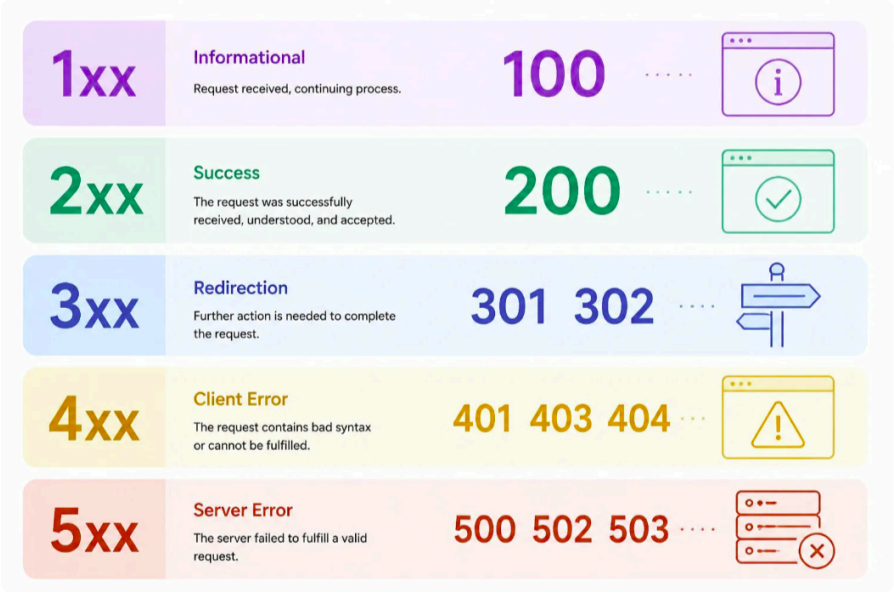
PUT → Update existing data

DELETE → Remove data



HTTP request steps

Status Codes



JSON

JavaScript Object Notation is a text-based data format. You and computers can both read it very easily (similar to Python).

Objects: { } and contain key-value pairs

Arrays: [ ] and contain ordered lists of values

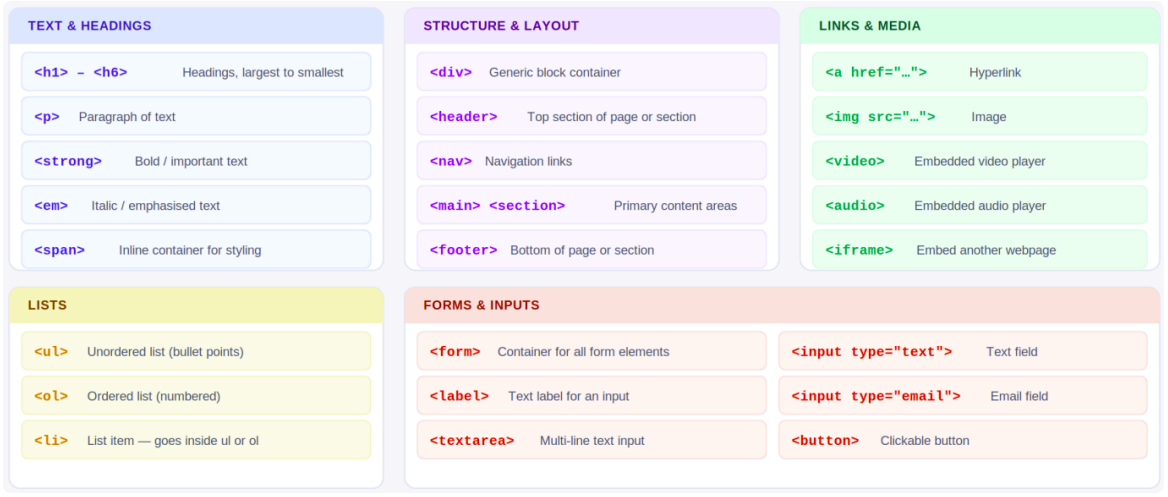
FLASK

15th of June (HC)

Frontend

HTML

- HyperText Markup Language
- There is an opening tag and closing tag



Common HTML tags